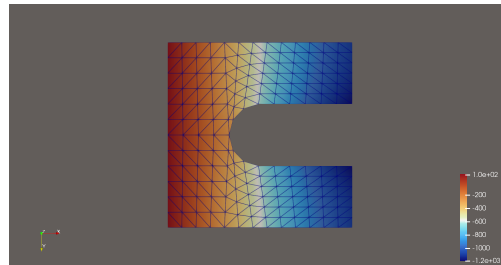


Projet de Recherche (PRe)

Spécialité : Mathématiques Appliquées

Année scolaire : [2024–2025]

Quantification d’Incertitude pour les Équations Différentielles Partielles



Auteur : PERRIN-DE LEUSSE Augustin
Promotion : [ENSTA 2026]

Tuteur ENSTA Paris : MOITIER Zois
Tuteur organisme d’accueil : KWOK Félix

Stage effectué du 02/06/2025 au 22/08/2025

Organisme d’accueil : GIREF Université Laval
Adresse : Département de mathématiques et statistique
1045 Avenue de la médecine
Université Laval
Québec, Canada

Mention de confidentialité

Ce document est non confidentiel. Il peut donc être librement communiqué et diffusé.

Remerciements

Lors de ce stage de recherche, j'ai eu l'honneur de rencontrer les différents membres du laboratoire du GIREF. J'ai par la même occasion pu travailler en étroite collaboration avec certains d'entre eux.

A ce titre, je souhaiterais remercier très chaleureusement Félix KWOK, mon maître de stage et chercheur au laboratoire du GIREF, pour sa chaleur humaine ainsi que sa clairvoyance et son aiguillage quant à mes travaux. Je remercie également Philippe-André Luneau, doctorant au laboratoire du GIREF qui m'a suivi dans mon travail au quotidien et sans qui ce rapport serait assurément moins fourni.

Je remercie également les autres stagiaires du département et de l'Université, qui par leur bonne humeur et les multiples expériences partagées sont devenus des amis, et qui se reconnaîtront en lisant ces lignes.

Résumé

Les travaux de ce stage ont porté sur l'étude des équations différentielles stochastiques. L'objectif principal est d'étudier comment intégrer de manière intelligente la quantification d'incertitude Uncertainty quantification dans un programme d'éléments finis.

L'incertitude aléatoire dans les caractéristiques d'un problème introduit une variable aléatoire dans les équations régissant ce même problème.

Un logiciel de résolution des équations différentielles - MEFPP, est développé dans le laboratoire du Groupe Interdisciplinaire de Recherche en Éléments Finis. Les solutions envisagées seront donc implémentées pour ce logiciel.

Nous considérerons tout d'abord la méthode directe de simulation Monte Carlo. Les variables d'entrées du problème sont tirées aléatoirement dans l'univers des réalisations, puis le problème est résolu par méthode des éléments finis, et la solution est stockée pour cette réalisation. Nous considérerons également une seconde méthode, la méthode Spectral Stochastic Finite Element Method, déjà étudiée et décrite dans la littérature scientifique. Cette méthode est une extension de la méthode des Éléments Finis, et permet directement d'obtenir une solution sous forme de variable aléatoire.

Ces deux méthodes seront implémentées via les outils présents dans le laboratoire, puis une analyse post-traitement sera menée pour pouvoir conclure quant à l'implémentation, et comparer les méthodes.

Abstract

The works of this research internship have been oriented toward the study of stochastic partial differential equations. The main goal is to study ways to implement the Uncertainty quantification Uncertaintyquantification in a finite element code.

In physics problems, the characteristics can be considerer random due to the uncertainties, thus leading to the solution of this problem being a random variable.

In the the laboratory where the internship has been conducted, the Groupe Interdisciplinaire de Recherche en Éléments Finis, has developped a finite element resolution software, called MEFPP. The studied solutions will be coded with MEFPP.

Firstly, we will be considering the Monte Carlo simulation method. The variables of the problem are drawn randomly in the possible realization universe, then the problem is solved using the Finite Element Method, and the solution associated with the realization is stored. We will then be considering the SS-FEM method, which has already been described in the scientific literature. This method is an extension of the Finite Element Method to the stochastic case. It allows to get a solution which is a random variable.

Both these methods have been developed with the tools of the laboratory, then a statistical analysis will be pursued in order to conclude regarding the implementation for both methods.

Table des matières

0.1	Introduction	2
0.1.1	Contexte	2
0.1.2	Présentation du laboratoire	3
0.1.3	Quantification d'incertitude et sujet de stage	3
0.1.4	Plan de réalisation du stage	4
0.2	Première Partie	6
0.2.1	Prise en Main	6
0.2.2	Problème Physique déjà étudié	7
0.2.3	Champ Aléatoire	8
0.2.4	Monte Carlo	9
0.2.5	Champ E aléatoire normal	11
0.2.6	Champ E aléatoire corrélé	12
0.2.7	Convergence Monte Carlo	15
0.2.8	Loi de U	15
0.3	SSFEM	17
0.3.1	Description SSFEM	17
0.3.2	Expansion KL	18
0.3.3	Expansion Polynomial Chaos	22
0.3.4	Plan de développement	23
0.3.5	Implémentation de la méthode	26
0.3.6	Validation de la méthode	28
0.4	Pistes à explorer et travaux futurs	32
0.5	Problèmes rencontrés	33
0.6	Conclusion	35
	Glossaire	37
0.7	Annexe	38
0.7.1	Visualisation des matrices c_{ijk} pour différents ordres, à troncatures KL et PC fixées.	38
0.7.2	Aberration de résolution lorsque $\text{ordrePC} \geq \text{ordreKL}$	39

0.7.3	Détail SSFEM	39
-------	------------------------	----

Table des figures

1	Maillage 3D tétraédrique du cylindre	8
2	Cisaillement $E = 200$	11
3	Ecart estimateur espérance pour E gaussien convolué, à x_{366} . .	15
4	test de kolmogorov, en abscisse la valeur des réalisation u , en ordonnée la densité de probabilité de réalisation.	17
5	Variance expansion KL troncature à 10 termes	21
6	Variance expansion KL troncature à 20 termes	21
7	Réalisation champ aléatoire KL pour $\mu = 0$	21
8	Exemple de l'algorithme dans le cas $M = 4, q = 2$, issu de [10] .	24
9	Solution déterministe cas $D(x, \theta) = 1$	28
10	Solution SSFEM ordreKL = 0, ordrePC = 0	29
11	Solution SSFEM ordreKL = 5, ordrePC = 1	29
12	Solution SSFEM ordreKL = 14, ordrePC = 10	30
13	Matrice des c_{ijk} pour $M = 3, P = 10, i = 0$	38
14	Matrice des c_{ijk} pour $M = 3, P = 10, i = 1$	38
15	Matrice des c_{ijk} pour $M = 3, P = 10, i = 2$	38
16	Solution SSFEM ordreKL = 5, ordrePC = 10	39

0.1 Introduction

0.1.1 Contexte

Ce stage porte sur l'étude des équations différentielles partielles stochastiques, dans un but de quantification d'incertitude. Il est réalisé dans un laboratoire universitaire spécialisé dans la Méthode des Éléments Finis (Groupe Interdisciplinaire de Recherche en Éléments Finis), faisant parti du département de mathématiques et statistiques de l'Université Laval.

Le laboratoire, travaillant historiquement avec Michelin, s'est intéressé il y'a 2 ans à la quantification d'incertitude. En collaboration, ils ont élaboré une liste d'idées à explorer à ce sujet. Dans ce contexte, les professeurs Félix KWOK et Jean DETEIX ont proposé un stage tourné vers ces problématiques. Au quotidien, le stage était supervisé par le doctorant Philippe-André Luneau.

Néanmoins, le laboratoire souhaitait avant tout offrir l'opportunité de réaliser un stage pédagogique et intéressant pour l'étudiant. Ainsi, dès le début, il a été mis au clair qu'il n'y avait pas d'objectif de réalisation particulier, et que les recherches et travaux étaient libres, et devaient surtout intéresser l'étudiant. La démarche scientifique et les problématiques ont donc été à l'initiative du stagiaire.

Il est à noter également que ce stage s'insère dans la continuité d'un projet, puisqu'un stage similaire avait été réalisé l'année dernière dans le même cadre et contexte, et avec les mêmes enjeux que ce stage. Il est donc nécessaire de décrire brièvement le travail effectué au préalable.

Le stagiaire précédent avait donc pris en main la Méthode des Éléments Finis, dont il avait codé une version en python dans un cas unidimensionnel et linéaire. Ensuite, un programme a été développé, à la fois sur MEFPP et en python, implémentant la méthode de simulation Monte Carlo sur un problème de cisaillement d'une poutre cylindrique horizontale. La poutre était fixée à une extrémité et soumise à une charge aléatoire à l'autre extrémité. Les résultats étaient exportés via python et post-traités sur Excel. Une implémentation de la méthode des perturbations avait également été envisagée, mais pas implémentée.

0.1.2 Présentation du laboratoire

Ce stage s'est effectué au sein du laboratoire du GIREF. Il s'agit d'un laboratoire de mathématiques spécialisé en Méthode numérique des Éléments finis. 3 Professeurs chercheurs permanents y sont affiliés : Jean DETEIX, Félix KWOK et José URQUIZA. En plus de cela, 4 doctorants y effectuent leurs thèses, 1 élève en maîtrise au Québec y effectue son projet et 4 stagiaires y ont effectué un stage cet été.

Une équipe de 3 développeurs fait également partie du laboratoire. Ils ont pour tâche le développement d'un outil d'éléments finis, notamment utilisé par les membres du laboratoire, ainsi que l'entreprise Michelin.

Le laboratoire travaille conjointement avec Michelin. Des réunions sont organisées de manière biannuelle, donnant lieu à des visites du GIREF à Michelin, en France, et vice-versa.

J'ai donc réalisé ce stage dans le laboratoire du GIREF, sous tutelle du professeur Félix KWOK et du doctorant Philippe-André LUNEAU. Ils avaient déjà encadré le stage effectué l'année précédente. Comme dit précédemment, l'orientation de ce stage n'étant pas fixée, le choix est resté très libre, et celui du stagiaire, qui a eu l'occasion de s'orienter vers ses domaines d'intérêt.

0.1.3 Quantification d'incertitude et sujet de stage

Présentons dans un premier temps le sujet sur lequel a porté ce stage, ainsi que l'intérêt et les enjeux dans lesquels ce sujet s'inscrit.

Les variables d'entrée et caractéristiques des problèmes physiques présentent généralement des incertitudes liées à des erreurs de mesure ou des faiblesses dans les processus. Ainsi, pour l'industriel notamment, il peut être intéressant d'anticiper l'impact que ces incertitudes auront sur la solution du problème. Il s'agit de la quantification d'incertitude (UQ). L'objectif est généralement de mener une analyse de probabilité de défaillance sur la solution, ou plus simplement une analyse statistique sur les moments de la solution [8].

Les travaux de ce stage s'appuieront sur un logiciel développé en interne appelé MEFPP (code d'éléments finis en C++, détaillé par la suite). Cet outil étant toujours en cours de développement, certaines contraintes d'implémentation devront être prises en compte lors de la réalisation de nos travaux.

L'incertitude aléatoire pour les équations différentielles peut être prise en compte de deux manières différentes dans un problème physique :

- Conditions aléatoires. Lorsque les conditions aux bords sont fonctions de

variables aléatoires. Dans des cas de déformation, cela constitue généralement un chargement aléatoire.

- Paramètres aléatoires. Lorsque les paramètres ou variables d'entrée de l'équation différentielle sont aléatoires. Dans le cas de déformation de matériaux, le module de Young est représenté comme un champ spatial aléatoire : $E(x, \theta), x \in \Omega, \theta \in \Theta$.

En étudiant ces équations, le but est généralement de réaliser une analyse post-traitement, sur les moments du premier ou deuxième ordre de la solution de ces équations. On peut aussi vouloir réaliser des études de probabilité de défaillance, qui, pour une limite fixée, nous donne des probabilités d'atteindre ou dépasser cette limite, en fonction des incertitudes d'entrée. Ce domaine d'étude présente donc des intérêts dans l'industrie et dans la conception de produits, en prenant en compte les incertitudes liées aux processus ou aux matériaux, et en analysant leurs impacts dans la chaîne de production et en sortie d'usine.

Des méthodes permettant ces analyses sont détaillées dans la littérature [8]. Nous nous intéresserons par la suite principalement à la méthode de simulation Monte Carlo et la méthode SSFEM. D'autres méthodes telles que la méthode des Perturbation, ou des méthodes hybrides existent également et sont détaillées dans la littérature [10].

0.1.4 Plan de réalisation du stage

Ainsi, le stage s'est découpé en 3 parties principales.

Lors de la première partie, qui a duré environ 3 semaines, le travail a été principalement un travail de lecture et prise en main. Plusieurs ouvrages ont été recommandés par le laboratoire, notamment sur les analyses statistiques pour l'UQ [8], et sur les méthodes Monte Carlo et SSFEM [10]. Un ouvrage écrit et publié au sein de l'université, de mathématiques et mécanique continue détaillant la méthode des Éléments Finis (sur laquelle est basée MEFPP) [4] a également été recommandé. Ainsi, les premières semaines ont été consacrées à la lecture de ces différents ouvrages. Il a été jugé pertinent de prendre en main le travail réalisé lors du stage précédent, synthétisé dans une présentation et des notebook. Finalement, il a également fallu prendre en main les outils MEFPP et GMSH.

La deuxième partie de ce stage, ayant durée également 2 semaines, a consisté en

la prise en main du problème physique et du code de simulation Monte Carlo sur lequel s'était appuyé le stage précédent. Une nouvelle bibliothèque interfaçant MEFPP et python ayant été conçue depuis mefpp4py, il a fallu transcrire les codes déjà réalisés, pour qu'ils utilisent cette bibliothèque, qui permet notamment de faciliter la compilation et le post-traitement des données.

Finalement, pendant les 9 semaines restantes, c'est-à-dire la majorité de ce stage, les travaux se sont portés sur la méthode SSFEM. Il a fallu dans un premier temps prendre en main cette méthode et les différentes méthodes mathématiques associées. Ensuite, il s'est agi d'implémenter cette méthode à l'aide de la bibliothèque mefpp4py, python, et MEFPP.

0.2 Première Partie

Lors des deux premières semaines de ce stage, il s'est agi de prendre en main les outils utilisés par le laboratoire.

0.2.1 Prise en Main

MEFPP

Avant toute chose, ce travail ayant été majoritairement réalisé à l'aide de MEFPP, il semble impératif d'en expliquer les fondements.

Mefpp est donc un logiciel de simulation numérique codé à l'intérieur du GIREF, utilisant la méthode des éléments finis [4] pour résoudre des problèmes dans des domaines variés. Il est compilé à l'aide d'un fichier de configuration ASCII (extension .champs), est interfacé avec PETSc (résolution algébrique) et MPI (exécution en parallèle). Il prend également en charge l'importation de maillage via gmsh, et l'exportation des résultats sous forme de fichier, notamment visualisables sur paraview (logiciel de visualisation).

Les principaux objets en MEFPP sont les champs, qui contiennent des valeurs en chaque degrés de liberté du problème. Ces champs peuvent être scalaires, vectoriels, constants ou interpolés sur chaque élément du maillage etc.

Un autre aspect important de MEFPP concerne ce qui est appelé des pré-post traitements pp. Il s'agit en quelque sorte de fonctions, qui sont appelées ainsi car elles sont généralement exécutées avant, pendant, ou après la résolution du problème d'éléments finis dans les fichiers .champ. Elles permettent de réaliser plusieurs choses telles que des exportations, des interpolations etc..

Ce logiciel est codé en C++ par les ingénieurs de recherche du laboratoire. Il est donc continuellement développé et de nouvelles fonctionnalités sont codées. Des réunions sont organisées bimensuellement pour faire l'état des lieux des progrès et des avancées de MEFPP. De plus, un effort particulier est fait pour la prise en main du logiciel, des tutoriels et documents explicatifs étant mis en place et à la disposition des membres du GIREF, contenant toutes les informations fondamentales sur MEFPP et permettant de compiler quelques problèmes physiques.

Ainsi, nous avons compilé certains problèmes, puis manipulé les objets et les fichiers de configuration afin de prendre en main ce logiciel. Les compilations

sont effectuées en SSH sur un ordinateur local au serveur du GIREF, possédant une version MEFPP locale. Puis les résultats étaient transférés et visualisés à l'aide de Paraview.

GMSH

GMSH est un logiciel de générateur de maillages aléatoires, qui peuvent servir à la résolution d'éléments finis. Ce logiciel permet de construire des géométries en 1D, 2D ou 3D, puis de générer des maillages en fonction de contraintes imposées. Il est ensuite possible de faire interagir ces maillages avec MEFPP pour résoudre les problèmes dans la géométrie définie.

Il est possible de générer des géométries à partir de l'interface graphique du logiciel, ou bien en codant un script en .txt, lu par le logiciel, et créant la géométrie étape par étape. Pour prendre en main ce logiciel, j'ai décidé de construire plusieurs géométries et maillages, notamment un pneu en 3D.

0.2.2 Problème Physique déjà étudié

Voici désormais le problème physique sur lequel s'effectuaient les simulations Monte Carlo lors du précédent stage.

Il s'agit donc d'un problème de cisaillement sur un cylindre élastique en 3 dimensions. La face gauche est fixée. La face droite est soumise à une contrainte, projetée aléatoirement.

Voici l'équation du problème de cisaillement :

$$\begin{cases} -\nabla \sigma(u) = 0 \\ u_{S_0} = 0 \\ \sigma(u) \cdot n = r = (\rho \sin(\phi) \cos(\theta), \rho \sin(\theta), -\rho \cos(\phi) \cos(\theta)) \end{cases}$$

Ainsi r était la contrainte aléatoire ; $\rho \sim \text{LogNormale}(-5,1)$, $\theta \sim \mathcal{N}(0, \frac{\pi^2}{576}\pi^2)$, $\phi \sim \mathcal{N}(0, \frac{\pi^2}{576})$.

σ est le tenseur de Cauchy du matériau, il dépend donc directement du module de Young puisque $\sigma = E \times \epsilon$ avec ϵ la déformation relative du matériau (d'après la loi de Hooke).

Il est à noter que le problème présente une symétrie de rotation autour de l'axe (Oyz) , est qu'il est donc équivalent de résoudre ce problème en 1D sur l'axe des

x.

L'aléatoire peut être considéré aussi bien dans les conditions au bord que dans les paramètres du problème. Le cas des paramètres aléatoires est mieux documenté pour la méthode SSFEM [10] [9]. De plus, le fait de considérer des variables aléatoires permet de rendre compte des inhomogénéités dans les propriétés physiques des matériaux. Pour ces raisons, nous considérerons désormais et dans le reste du stage des paramètres d'entrée du problème comme aléatoires. Dans le problème de cisaillement, cela se traduit donc par un module de Young comme champ aléatoire.

$$\begin{cases} -\nabla \sigma(u) = 0 \\ u_{S_0} = 0 \\ \sigma(u) \cdot n = 0 \\ E(x, \theta), \theta \in \Omega \end{cases}$$

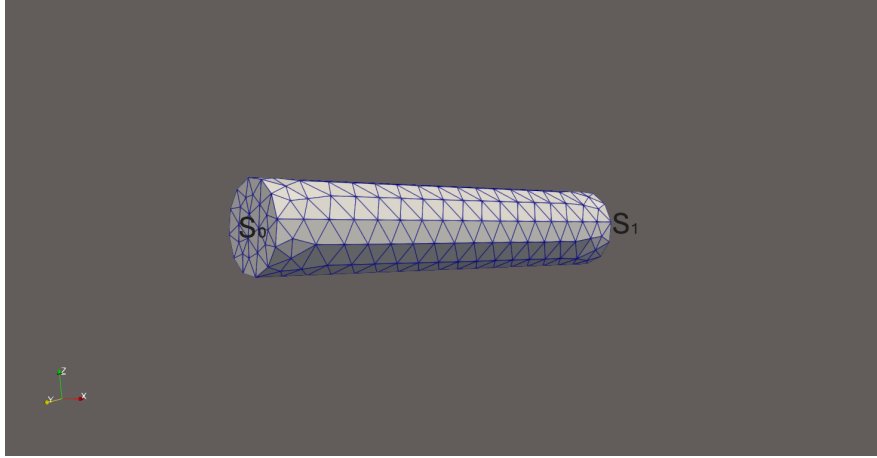


FIGURE 1 – Maillage 3D tétraédrique du cylindre

0.2.3 Champ Aléatoire

Un champ spatial aléatoire est une fonction $H(x, \theta)$ avec $x \in \Omega$ l'espace du domaine de définition du problème, c'est-à-dire la géométrie. Dans ce cas, il s'agit de l'espace à l'intérieur du cylindre. θ est une variable aléatoire tiré dans l'univers des réalisations Θ .

Dans la méthode des éléments finis, il est donc nécessaire de discrétiser $H(x), x \in$

$\Omega \rightarrow \hat{H}(x_i), i \in [0, N]$. Il existe plusieurs méthodes de discrétisation : [10] :
 Pour chacune de ces méthodes, considérons préalablement que l'espace Ω est maillé en sous-espaces élémentaires Ω_e tels que $\bigcup_e \Omega_e = \Omega$.

- Mid Point (MP) : On considère $H(x) = H(x_c)$, $x \in \Omega_e$, et x_c le point au centre de Ω_e
- Shape Function (SF) : On considère : $\hat{H}(x) = \sum_{i=1}^q N_i(x)H(x_i)$ avec $x \in \Omega_e$, x_i le i ème noeud de Ω_e et N_i des fonctions polynomiales. Cette représentation permet d'avoir un champ H continu sur Ω .
- Integration Point : On associe des points de gauss à chaque Ω_e , et pour chacun de ces points, on associe $\hat{H}(x_i) = H(x_i)$.

Dans notre cas, nous utiliserons MEFPP pour discrétiser spatialement à l'aide de la méthode Integration Point.

Dans le cas du problème de cisaillement d'une poutre, nous considérerons successivement trois cas :

- Déterministe, $E = 200$ constant sur l'espace.
- Aléatoire normal ; nous considérerons qu'en chaque noeud du maillage, E suit une loi normale, $E(x_0) \rightarrow \mathcal{N}(200, 10)$, puis MEFPP interpole E entre les noeuds
- Aléatoire normal puis convolué spatialement. C'est une extension de la représentation précédente.

0.2.4 Monte Carlo

Nous avons donc codé un script python utilisant la bibliothèque mefpp4py, capable de réaliser des simulations Monte Carlo pour le problème de cisaillement, avec un module de Young aléatoire.

Le code MEFPP de cisaillement élastique était déjà développé lors de mon arrivée, ainsi que la géométrie du problème, mais n'était pas compatible avec mefpp4py. Ainsi a-t-il fallu le modifier quelques peu. Une classe a ainsi été développée en Python. Voici les méthodes de cette classe :

- `init` Il est tout d'abord nécessaire de charger le fichier de configuration `.champs`, puis de récupérer le champ E initialisé sur MEFPP, ainsi que le champ vectoriel de déplacement U , solution du problème. Cette méthode récupère également les différents pp, initialisés en MEFPP (résolution du problème, exportation, copie des vecteurs en python vers les champs en

MEFPP).

- **Monte_Carlo**. Réaliser une simulation Monte Carlo, avec en paramètres le nombre de simulation à réaliser, et stockant chaque résultat dans un attribut de type liste de la classe.
- **Traitement** De traiter les résultats obtenus par Monte Carlo, en calculant les moyennes et variances des déplacements obtenus
- **Affichage_traitement** D’afficher les résultats du traitement obtenus
- **Critère_Cov** De calculer le nombre minimum de simulations à lancer pour respecter le critère de COV. Le critère de COV est défini comme suit : $COV = \frac{\sqrt{\text{Var}[U]}}{\sqrt{NE[U]}}$. Pour chaque simulation, $\max_{U_i}()$ COV est calculé, et si ce maximum est inférieur à une limite (en l’occurrence 0.05), on conclut que la simulation a convergé en N itérations. Un critère COV de convergence de la simulation Monte Carlo est généralement situé entre 0.05 et 0.01.
- **Affichage_realisation** D’afficher un histogramme des réalisations des déplacements en un noeud fixé.

Ce code a donc été utilisé pour étudier les trois cas susmentionnés.

Champ constant

Le problème a tout d’abord été résolu à module de Young $E = 200$ constant. Le cylindre est de taille $l = 10$ sur l’axe Ox , et nous observons :

$$E = 200, \quad \sigma = 5, \quad , u_{S_1} = 0.248$$

La loi de déformation de Hooke est donc bien vérifiée.

$$\sigma = \frac{Eu_{S_1}}{l}$$

L’échelle en bas à droite (Figure 2) correspond à la valeur de la norme du déplacement u . La figure correspond aux noeuds du maillage du cylindre. On observe donc un gradient de déformation, $||u|| = 0$ sur la face gauche et $||u|| = 0.25$ sur la face droite.

Nous réalisons donc une analyse de convergence de la méthode, en étudiant le critère COV.

On observe que ce critère est très largement respecté, puisqu’après seulement deux simulations, on atteint la tolérance $1e-14 \ll 0.05$. C’était attendu puisque l’équation est déterministe, la variance et donc COV étant ainsi nuls. Néanmoins, les COV ne sont pas strictement nuls. Cela peut sans doute être imputé aux approximations numériques de l’ordinateur.

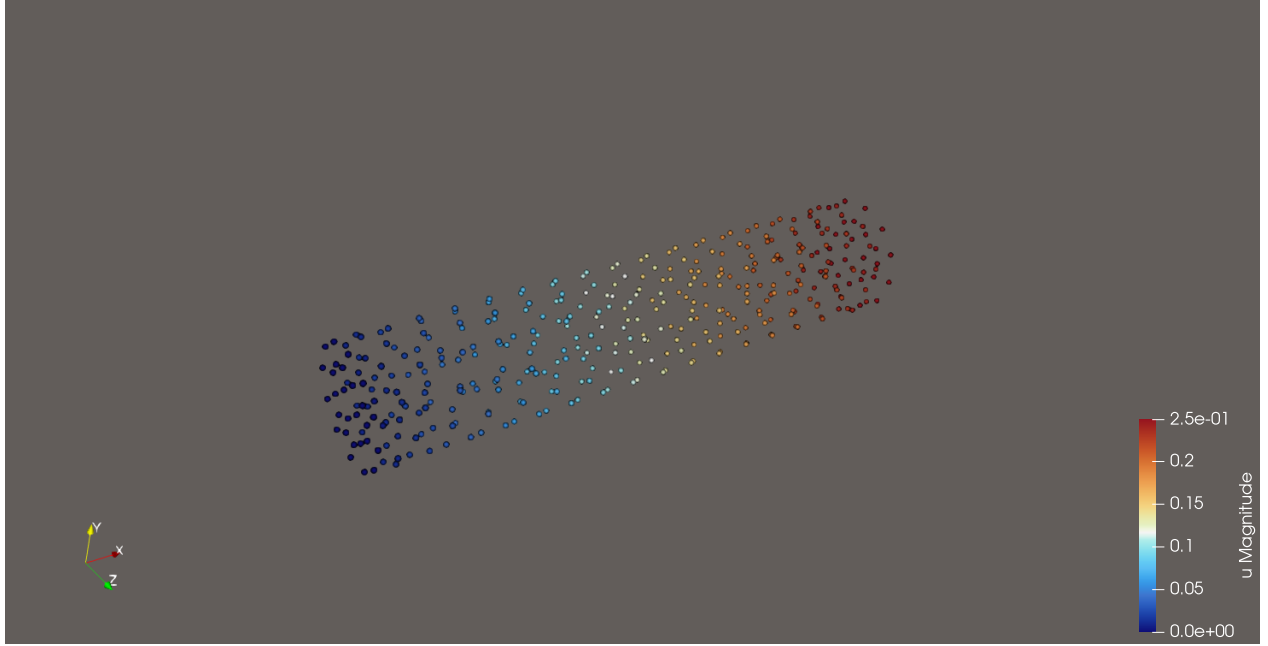


FIGURE 2 – Cisaillement $E = 200$

0.2.5 Champ E aléatoire normal

Nous considérons désormais $E(x_i) \sim \mathcal{N}(200, 10)$, avec $(x_i)_{i \in [1, N]}$ les coordonnées des noeuds du maillage. E est interpolé entre les noeuds du maillage.

Il n'existe pour le moment pas de fonctions en MEFPP capable de représenter directement un champ (en l'occurrence E) de manière aléatoire. Ainsi, l'implémentation a été envisagée de cette manière :

Côté MEFPP

- Création d'un champ E déterministe
- Création d'une variable aléatoire $\theta \rightarrow N(200, 10)$
- Création d'une fonction d'interpolation capable d'interpoler une variable sur un champ

Côté Python :

- Référencement à la variable aléatoire
- Référencement à la fonction d'interpolation
- Appel de la fonction d'interpolation Ainsi, le champ E était finalement tiré aléatoirement sur chaque noeud du maillage, pour chaque résolution effectuée.
- Copie de la matrice E en python dans l'objet champ E en mefpp.

Analyse

On utilise le critère $\text{COV} \leq 0.0l$ pour analyser la convergence de la méthode Monte Carlo.

- Pour $l = 0.01$, convergence de la méthode en 9 itérations
- Pour $l = 0.001$, la loi des grands nombres nous dit qu'il faut multiplier le nombre d'itérations par 100. On s'attend donc à devoir réaliser 900 itérations en ordre de grandeur. La simulation a été lancée jusqu'à 200, et on obtient $\text{COV}(m = 200) = 0.00154$, ce qui semble donc cohérent, par interpolation linéaire.

0.2.6 Champ E aléatoire corrélé

Désormais, nous nous intéressons à un champ aléatoire E défini par sa matrice de covariance spatiale, $\text{Cov}(E(x'), E(x)) = f(x', x)$.

Une méthode possible est la factorisation de Cholesky. Néanmoins, elle nécessite plusieurs étapes de manipulation des champs sous forme de matrices. Il est possible de l'implémenter à l'aide de `mefpp4py`, mais en considérant que cette bibliothèque connaît un développement rapide, qui prendra potentiellement en charge cette méthode dans le futur, nous nous sommes intéressés à une différente manière de construire un champ aléatoire à covariance déterminée. Il s'agit de la méthode de convolution spatiale.

Convolution

Voici les principes de la convolution. [1]

Soit l'espace Ω définissant la géométrie du problème. Soit le noyau (ou fonction)

$$k : \Omega \rightarrow \mathbb{R}, \quad s \mapsto k(s).$$

Soit également un bruit blanc, y , tel que $\forall x \in \Omega, \quad y(x) \sim \mathcal{N}(0, 1)$. Alors la convolution de y par le noyau k est la fonction z :

$$z(s) = \int_{\Omega} k(u - s) y(u) du, \quad \text{pour } s \in \Omega.$$

$\forall s \in \Omega, z(s)$ est un processus Gaussien. Il est de plus possible de calculer la fonction de covariance de z entre deux points s et s' :

$$d = s - s', \quad c(d) = \text{Cov}(z(s), z(s')) = \int_{\Omega} k(u - s) k(u - s') du = \int_{\mathbf{S}} k(u - d) k(u) du.$$

Ainsi, on observe un rapport de transformée de Fourier entre la convolution c et le noyau k . Plus précisément, [1] :

$$k(s) \xrightarrow{\mathcal{F}} K(\omega) \xrightarrow{\cdot^2} K^2(\omega) \xrightarrow{\mathcal{F}^{-1}} c(s)$$

$$k(s) \xleftarrow{\mathcal{F}^{-1}} C^{1/2}(\omega) \xleftarrow{\sqrt{\cdot}} C(\omega) \xleftarrow{\mathcal{F}} c(s)$$

Spécifions maintenant un noyau gaussien : $k(s) = \exp(-s^2)$. Alors par les relations précédentes, on obtient $c(d) = \exp(\frac{-d^2}{2})$.

Il est à noter que par le Lemma 1 de [5] nous trouvons le même résultat. Ce résultat est d'ailleurs directement décrit, sans démonstration, dans [6] également.

Dans le cas des méthodes numériques, nous ne travaillons non pas avec un espace continu mais avec un maillage. Ainsi l'opérateur de convolution devient une quadrature de Gauss de l'opérateur continu. Soit les $(s_i)_{i \leq N}$ sommets du maillage :

$$z(s) = \frac{\sum_{i=1}^N k(s - s_i)x(s_i)}{\sum_{i=1}^N k(s - s_i)}$$

La convolution peut également dépendre d'un paramètre r appelé fenêtre. Pour r et s fixés, on ne convolve qu'avec les voisins s_i tels que $\|s - s_i\| \leq r$.

Code Python de la Convolution

Ainsi, nous souhaitons dans un premier temps construire en python un champ aléatoire spatial convolué, dont la fonction de covariance est $Cov(E(s), E(s')) = \exp \frac{-s^2}{2}$, à partir d'un bruit blanc. A cet effet, un notebook a été codé, car la facilité de visualisation et le code par blocs semblaient être pertinents pour cette implémentation. Voici la description de la démarche de réalisation de ce code :

- Nous commençons par implémenter la méthode de Cholesky, qui prend en entrée la fonction de covariance et un bruit blanc, et qui retourne en sortie une réalisation du champ aléatoire E désiré.
- Ensuite, nous codons la méthode de Convolution, qui prend en entrée la fonction de covariance et le noyau de convolution et qui retourne en sortie une réalisation du champ aléatoire
- Nous codons les méthodes de Monte Carlo associées à chacune de ces deux méthodes de construction du champ aléatoire

- Nous codons enfin les méthodes calculant l'estimateur du maximum de vraisemblance de la matrice de covariance, qui prend en entrée une liste de réalisations du champ aléatoire construit par Cholesky ou Convolution, et qui retourne une matrice de taille $N \times N$.

De cette manière, nous sommes capables de comparer les matrices de covariance (estimateurs de maximum de vraisemblance) des champs aléatoires obtenus par Cholesky et par Convolution.

Ces matrices ont été calculées pour $N \in [2, 10]$, avec 10000 simulations Monte Carlo. Expérimentalement, en fixant le noyau de convolution $k(s) = \exp(-s^2)$, on constate $C(d) = \exp(-\frac{s^2}{4})$. Il y a donc un écart d'un facteur 1/2 par rapport au résultat théorique attendu. Malheureusement, à ce jour, nous n'avons pas encore trouvé le problème, qui doit sûrement être du à l'implémentation informatique de la convolution.

La covariance trouvée étant toutefois exponentielle, pour la suite, nous nous sommes contentés de ce résultat. L'écart entre les matrices est d'environ 5%, $\forall N \in [2, 10]$ en moyenne (ce que nous avons considéré négligeable).

Nous souhaitons désormais construire un champ E par convolution, utilisable dans MEFPP, puis réitérer la démarche réalisée pour le cas d'un champ aléatoire sur les noeuds. Fort heureusement, des codes de filtrage par convolution des champs de MEFPP, codé en C++, ont déjà été implémentés. Ainsi, il suffit uniquement de modifier la fonction noyau pour obtenir le champ souhaité.

Nous avons une nouvelle fois considéré $\mathbb{E}[E] = 200$, ainsi que $\text{Var}E = 10$, c'est-à-dire en prenant $\text{Cov}(E(x), E(x')) = 10 \exp(-\frac{s^2}{4})$.

Analyse

On utilise le critère $\text{COV} \leq l$ pour la convergence de la méthode de Monte Carlo.

- Pour $l = 0.01$, convergence de la méthode en 3 itérations.
- Il est à noter que la convergence est d'autant plus rapide que le rayon de convolution est grand. En effet, plus le rayon est grand, plus les valeurs aléatoires extrêmes sont pondérées et se rapprochent d'une moyenne, à savoir $E_0 = \mathbb{E}[E]$.

Il est attendu que la convergence soit plus rapide pour le champ corrélé, comparativement au champ tiré aléatoirement, puisque les valeurs éloignées de la moyenne sont lissées par la convolution. Néanmoins, cela est à nuancer puisque

dans le cas aléatoire, le champ tiré aléatoirement sur chaque noeuds du maillage était ensuite interpolé sur les éléments, ce qui revient à lisser le champ. Tandis que le champ convolué était implémenté comme constant sur chaque élément élémentaire Ω_e du maillage.

0.2.7 Convergence Monte Carlo

Nous souhaitons valider le fait que l'estimateur de l'espérance de u converge effectivement vers $u_{\text{déterministe}}$, dans le cas d'un champ aléatoire normal, et d'un champ convolué.

Nous avons cherché les points de maximum de variance dans les deux cas :

- Pour aléatoire normale : point numéroté 304, soit $x = (10, -0.858497, 0.512815)$ sur le maillage.
- Pour convolution gaussie : point numéroté 366, soit $x = (10, 0.617285, -0.171119)$ sur le maillage.

Ces points sont tous les deux sur la face droite S_1 du cylindre. Comme attendu,

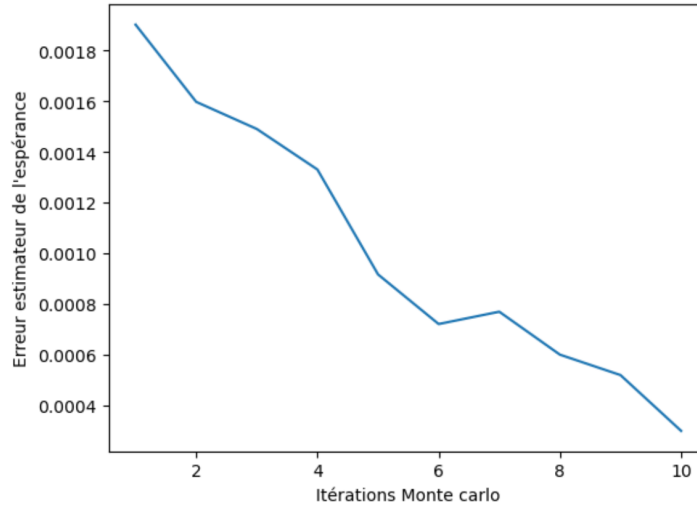


FIGURE 3 – Ecart estimateur espérance pour E gaussien convolué, à x_{366}

les estimateurs de l'espérance de u semblent converger vers u déterministe.

0.2.8 Loi de U

Nous souhaitons désormais valider la loi de probabilité que suit effectivement U , en effectuant un post-traitement statistique. Voici les deux méthodes envisageables :

- Test du χ^2 : non-adaptée au cas continu.
- Test de Kolmogorov-Smirnov.

Le test de Kolmogorov étant applicable aux problèmes continus, c'est celui-ci qui étudié.

Test de Kolmogorov

Ce test consiste à comparer la loi que semble suivre un échantillon avec une loi théorique définie au préalable. Elle permet de valider l'hypothèse : (H_0) : La loi expérimentale suit la loi théorique. Voici les étapes de la méthode :

- Calcul de la fonction indicatrice de l'échantillon par quadrature de gauss (ici méthode des rectangles).
- Calcul de la distance maximum entre la fonction indicatrice théorique et expérimentale
- Comparaison de cette valeur avec une table théorique fonction de la taille de l'échantillon.

Nous avons donc implémenté une classe `ks` en python, prenant en paramètre l'échantillon théorique tiré, ainsi qu'une loi donnée, et retournant notamment le résultat du test, à savoir la validation ou non de l'hypothèse (H_0) .

On a considéré la norme de la solution en un point arbitraire (le 1000ème noeud du maillage). On a calculé la moyenne et variance expérimentale pour 40 simulations monte carlo. On a ensuite comparé avec la loi normale de même moyenne mais de variance 50 fois plus grande. Voici le fit obtenu : En comparant avec la table, l'hypothèse de fit est validée. Ainsi, par Monte Carlo, l'estimateur de l'espérance de la solution est égal à la solution déterministe, dans le cas où $E = 200$. Néanmoins, la variance spatiale de E se traduit en une variance spatiale de U 50 fois plus grande. Cette valeur n'a pas encore été expliquée théoriquement.

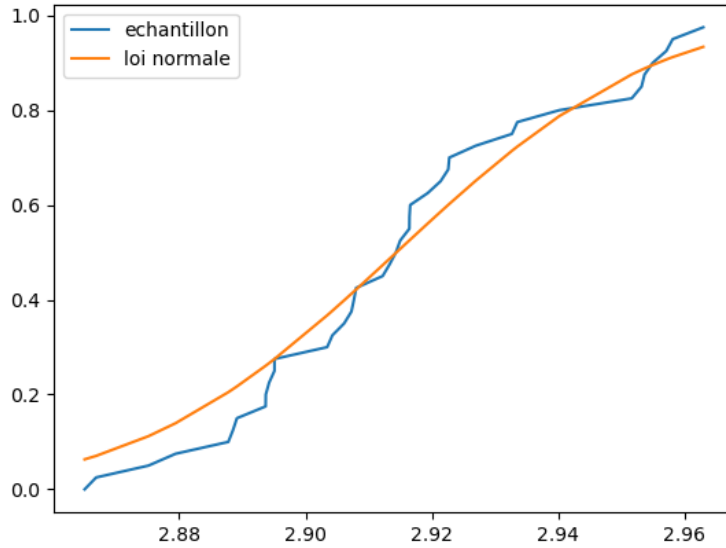


FIGURE 4 – test de kolmogorov, en abscisse la valeur des réalisation u , en ordonnée la densité de probabilité de réalisation.

0.3 SSFEM

Dans la seconde partie de ce stage, nous nous sommes donc intéressés à la Méthode Spectrale Stochastique des Éléments Finis. Il s'est agi d'appréhender, puis implémenter cette méthode pour MEFPP et python. Voici dans un premier temps une description de cette méthode.

0.3.1 Description SSFEM

La méthode SSFEM est donc une extension de la méthode FEM pour les problèmes aux propriétés matérielles stochastiques.

Considérons un système physique. Ce système est régi par des équations différentielles partielles et des conditions au bord. Les méthodes de résolution numérique de ces problèmes fonctionnent à l'aide d'une discrétisation spatiale du problème.

Cependant, si une propriété matérielle du problème tel que le module de Young ou le champ de conductivité, est un champ aléatoire, alors le système est régi par un ensemble d'équations différentielles stochastiques. Supposons que le champ de déplacement est solution du problème. Nous aurons donc $u(x, \theta)$ avec x la

coordonnée spatiale, et θ une réalisation dans l'univers des réalisations Θ . Dans ce cas, le problème pourra être résolu en discrétisant à la fois dans le domaine spatial, ainsi que dans l'univers des réalisations Ω .

Il y a deux procédures de discrétisation supplémentaires à considérer dans la méthode SSFEM :

- La première est la discrétisation de la propriété matérielle aléatoire à l'aide de l'expansion de Karhunen-Loève.
- La deuxième est la discrétisation de la solution u sur une base de l'espace des variables aléatoires $\mathcal{L}^2(\Theta, \mathcal{F}, \mathcal{P})$, à savoir la "polynomial chaos" PC.

Ainsi, la finalité de la méthode est l'obtention d'un vecteur de coefficients sur une base (tronquée) de $\mathcal{L}^2(\Theta, \mathcal{F}, P)$, sur lesquels il est possible d'effectuer des analyses du premier, second ordre, des analyses de probabilité de défaillances ou autre.

0.3.2 Expansion KL

Détaillons ici l'expansion KL mentionnée plus tôt.

L'expansion KL est une méthode de représentation en série d'un champ spatial aléatoire, à partir de la décomposition spectrale de la covariance de ce même champ. Soit $E(x, \theta)$ le champ aléatoire. En notant $\text{Cov}(x, x')$ la fonction de covariance du champ, puisque cette fonction est symétrique définie positive et bornée, il est possible de résoudre le problème de valeur propre (intégrale de Fredholm) :

$$\forall i = 1, \dots \quad \int_{\Omega} \text{Cov}(x, x') \varphi_i(x') d\Omega_{x'} = \lambda_i \varphi_i(x)$$

Où les $\{\lambda_i\}_i$ et $\{\varphi_i\}_i$ sont les inconnues.

Les fonctions $\{\varphi_i\}_i$ forment une base de $\mathcal{L}^2(\Omega)$. A partir des coefficients λ_i et φ_i , nous pouvons étendre le champ tel que :

$$E(x, \theta) = \mu(x) + \sum_{i=1}^{\infty} \sqrt{\lambda_i} \varphi_i(x) \xi_i(\theta)$$

avec les ξ_i des variables aléatoire orthonormales, $\sim \mathcal{N}(0, 1)$.

On observe que $\mu(x)$ est la partie déterministe ou l'espérance (indifféremment) du champ aléatoire.

Par définition de la covariance, symétrique, bornée et définie positive, Cov admet

la décomposition spectrale :

$$\text{Cov}(x_1, x_2) = \sum_{n=0}^{\infty} \lambda_n f_n(x_1) f_n(x_2)$$

avec λ_n et f_n les valeurs et vecteurs propres du noyau de covariance. C'est-à-dire :

$$\int_D C(\mathbf{x}_1, \mathbf{x}_2) f_n(\mathbf{x}) d\mathbf{x}_1 = \lambda_n f_n(\mathbf{x}_2)$$

[9] Explique le lien entre l'équation de Fredholm et l'expansion en série de Karhunen Loève.

A partir de cette expression, il est possible de calculer la covariance et la variance du champ aléatoire ainsi obtenu par l'expansion KL :

$$\text{Cov}(x_1, x_2) = \sum_{i=1}^{\infty} \lambda_i \varphi_i(x_1) \varphi_i(x_2)$$

$$\text{Vaf}(E(x)) = \sum_{i=1}^{\infty} \lambda_i \varphi_i(x)^2$$

Pour des raisons computationnelles, il est nécessaire de tronquer cette série à l'ordre M , que nous appelons dans la suite ordre KL. [9] Explique comment calculer l'erreur de cette troncature.

Le problème principal de l'expansion est la résolution de l'équation de Fredholm. Il est donc utile de choisir une fonction de covariance particulière, telle que la résolution du problème de valeur propre soit déjà considérée et étudiée dans la littérature. Il est à noter qu'il est également possible de résoudre numériquement cette équation.

Résolution de l'équation de Fredholm

Nous considérons donc à cet effet la fonction gaussienne $\exp(-\frac{(x-y)^2}{2w^2}) = \text{COV}(x, y)$ comme fonction de covariance de K .

Il est possible de résoudre l'équation de Fredholm pour ce noyau en trouvant une solution ansatz, et en prouvant que les inconnues $\{\lambda_i\}_i$ et $\{\varphi_i\}_i$ vérifient l'équation de Fredholm pour cette fonction gaussienne. Il est possible de démontrer cela par la formule de Mehler [7] ou bien en utilisant des relations d'orthogonalité des polynômes de Hermite [2]. Le calcul n'a pas été mené dans le cadre du

stage.

Ainsi, les solutions sont [7] :

$$\varphi_n(x) = ce^{-\frac{x^2}{2\alpha^2}} h_n\left(\frac{x}{\beta}\right)$$

$$\left\{ \begin{array}{l} \frac{\alpha}{w} = \sqrt{\frac{1}{1-r}} \\ \frac{\beta}{w} = \sqrt{\frac{r}{1-r^2}} \\ c = \left(\frac{1+r}{1-r}\right)^{\frac{1}{4}} \\ \lambda_n = \frac{r^n}{1-r} \end{array} \right.$$

avec $r \in [-1, 1]$, supposément un paramètre libre, h_n les polynômes de Hermite probabilistes. La série obtenue est tronquée à l'ordre N.

Implémentation expansion KL

Cette expansion a donc été implémentée dans une classe python, dans le cas d'un champ aléatoire à covariance de type gaussienne normalisée.

Pour vérifier l'implémentation, la première étape est de calculer la variance du champ sur l'espace. Puisque la covariance est gaussienne normale, nous souhaitons avoir $\forall x \in \Omega, \text{Var}(E(x)) = 1$. Nous comparons le covariogramme du champ obtenu avec celui obtenu par Cholesky et par convolution.

Nous avons visualisé des réalisations du champ aléatoire ainsi étendu et tronqué. En comparant ces visualisations avec des réalisations du même champ obtenues par la méthode de Cholesky ou par Convolution, nous observons que les champs semblent similaires.

Nous observons que plus l'ordre de troncature augmente, plus le domaine où la variance est proche de 1 augmente. Qui plus est, sur le "domaine plat", la convergence vaut 1, à une erreur de $1e - 5$ près.

Cette réalisation d'un champ aléatoire centré en 0 (Figure 7), montre que nous

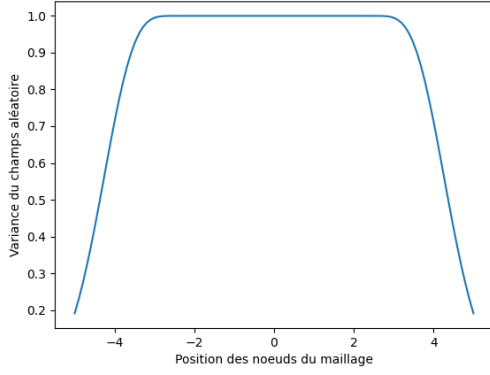


FIGURE 5 – Variance expansion KL
troncature à 10 termes

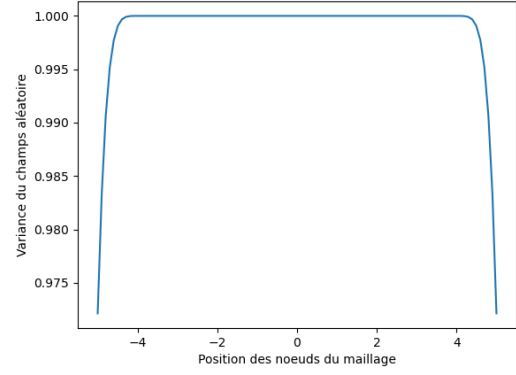


FIGURE 6 – Variance expansion KL
troncature à 20 termes

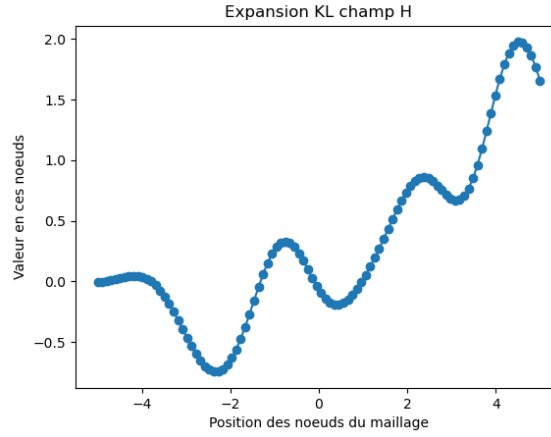


FIGURE 7 – Réalisation champ aléatoire KL pour $\mu = 0$

obtenons en effet un champ spatialement corrélé. Ces réalisations ont les mêmes caractéristiques que pour le cas Cholesky.

Le code de calcul des estimateurs de matrice de covariance a été repris pour analyser les réalisations Monte Carlo de l'expansion KL. Ainsi, les estimateurs pour la méthode KL et la méthode Cholesky présentent une erreur relative inférieure à 5%, pour $n = 1000$ simulations Monte Carlo. Nous en avons donc conclu que l'implémentation était valide. L'expansion est valide et convergente pour toute valeur de r , mais la vitesse de convergence semble varier avec r . Ainsi, nous considérerons pour le moment $r = 0.26$ qui semble présenter un maximum de vitesse de convergence.

0.3.3 Expansion Polynomial Chaos

Détaillons désormais le Polynomial Chaos, et l'implémentation qui en a été faite.

Nous nous intéressons dans un second temps au Polynomial Chaos. En essence, il s'agit de construire une base de polynômes de l'espace des variables aléatoires $\mathcal{L}^2(\Theta, \mathcal{F}, P)$. Cette décomposition permet donc de représenter des fonctions de variables aléatoires dans cette base. Dans le cas de la méthode SSFEM, nous utilisons cette expansion pour représenter notre solution :

$$T = \sum_{j=0}^{\infty} \mathbf{T}_j \psi_j(\theta)$$

avec les $(\psi_j)_j$ les fonctions de la base Polynomial Chaos.

Pour les mêmes raisons computationnelles, il est nécessaire de tronquer cette expansion. Il faut tout d'abord considérer la troncature de cette expansion, au terme P , que nous appellerons par la suite ordre PC. Par analogie avec la représentation en série de Fourier, lorsque notre champ présente des "irrégularités", il faut considérer un ordre de troncature important pour pouvoir conserver ces "irrégularités".

Les polynômes de PC sont classés par leur ordre. La base sera définie par 2 paramètres :

- La "dimension" M , qui correspond à la taille de chaque variable aléatoire $\xi_i(\theta)$. Il est à noter que cette variable aléatoire est liée aux variables aléatoires de l'expansion KL.
- L'ordre de troncature P . Sans rentrer trop dans le détail, il est nécessaire d'implémenter les P premiers polynômes PC, qui présentent des ordres q croissants. Il y a plusieurs polynômes PC du même ordre, en fonction de M .

Code du Polynomial Chaos

Pour coder la polynomial chaos, nous avons utilisé la même méthode que dans [10]. Voici donc la méthode pour construire les polynômes d'ordre q de PC :

Nous avons $M+q-1$ boîtes, et $M-1$ balles à placer. Chaque boîte peut contenir une ou zéro balle. A l'instant initial, toutes les boîtes les plus à gauche sont remplies. Nous avons donc une configuration initiale, que nous ajoutons à la liste des configurations. L'algorithme est le suivant :

- Nous prenons la première balle, en partant de la droite, contenant une case vide, et la déplaçons d’une boîte vers la droite. Toutes les balles à droite de la balle déplacée sont ramenées directement à droite de la balle déplacée.
- A chaque étape de l’algorithme, la configuration courante est ajoutée à la liste des configurations.
- Lorsque toutes les balles sont dans les boîtes les plus à gauche, l’algorithme est terminé. Toutes les configurations de la liste des configurations sont converties en une séquence d’entiers $\{\alpha_i, i = 1, 2, \dots\}$. Cet ensemble décrit tous les polynômes d’ordre q et de variable de dimension M de PC.

Ainsi, cette méthode a été implémentée dans un fichier python (pas une classe cette fois). La classe finalement codée doit donc être capable de calculer les coefficients c_{ijk} nécessaires à la minimisation du résidu. Voici les fonctions implémentées, dans le fichier `p_chaos.py` :

- `find_last_one` : trouve et retourne la position de la balle la plus à droite dans une configuration donnée.
- `recursif` : fonction récursive permettant de générer toutes les configurations possibles des balles, pour un M donné et un degré des polynômes q donné.
- `traduction` : fonction traduisant une configuration de balles en la séquence d’entier correspondante.
- `alpha_gras` : fonction calculant tous les coefficients α définissant les polynômes de la base PC, pour un M donné et un nombre de polynômes à calculer donné.
- `calcul_cijk` : fonction calculant le coefficient c_{ijk} pour i, j, k donné.
- `matrice_cijk` : fonction calculant la matrice $(C_i)_{jk} = c_{ijk}$ pour un i donné. Cette fonction permet de visualiser les calculs c_{ijk} et donc l’apport de chaque matrice de rigidité K_i à la matrice finale $\mathcal{K}$. Elle n’est pas utilisée dans les autres codes, mais a été développée à des fins de débugges notamment.

0.3.4 Plan de développement

La méthode se déroule donc en plusieurs étapes. Par soucis d’organisation et de clarté, nous avons abordé l’implémentation informatique de cette méthode à l’aide d’un plan de développement. L’objectif est donc de synthétiser chaque étape d’implémentation, nécessaire à la méthode, pour pouvoir avancer étape par étape. Voici le plan successif :

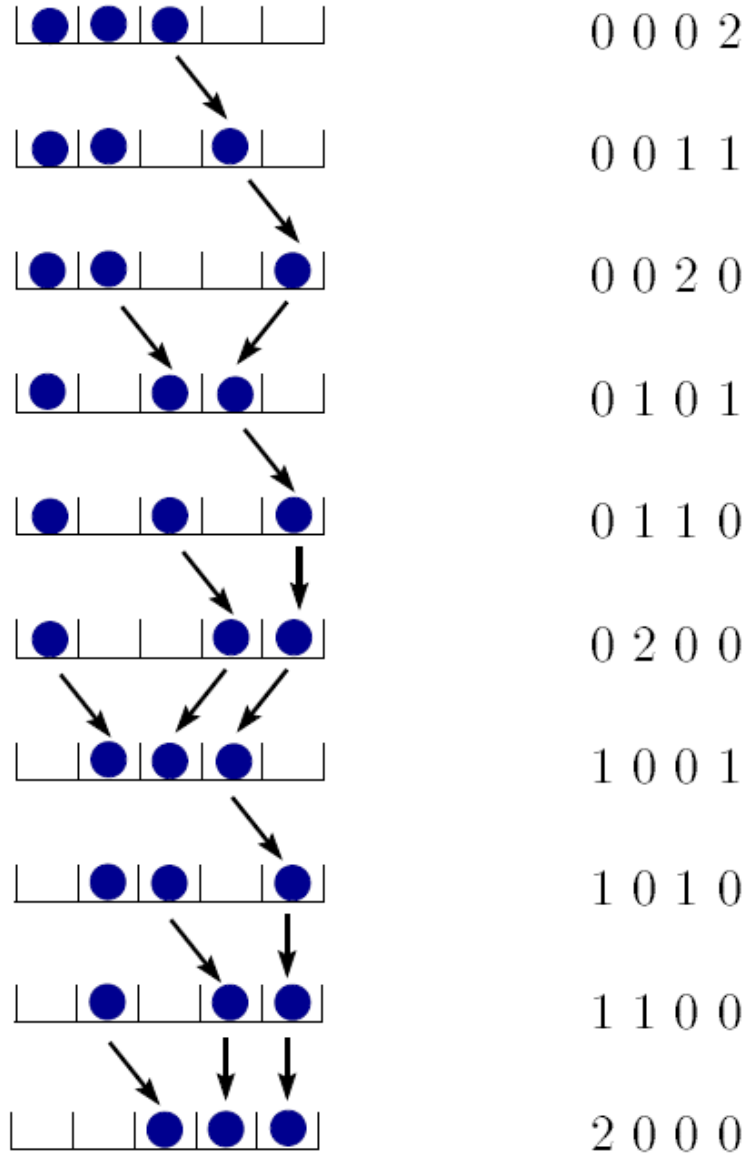


FIGURE 8 – Exemple de l’algorithme dans le cas $M = 4, q = 2$, issu de [10]

- MEFPP
 - Création du maillage du problème physique en gmsh, puis conversion en .mail, .enti
 - Code du problème physique dans le fichier de configuration .champs
- KL : Ordre, r
 - Code des $\sqrt{\lambda_i}$ en python
 - Code des φ_i sur `mefpp`
 - Implémentation des fonctions φ_i sur python
 - Lire ligne dynamiquement de python à mefpp
 - Récupération des $\sqrt{\lambda_i}$ sur `mefpp`
 - Calcul et assemblage de la matrice de rigidité $\mathbf{K}_i$
 - Calcul et assemblage du vecteur de résidu $\mathbf{F}$
- P.C : Ordre, dimension
 - Code de l'algorithme de définition des ψ_j [10]
 - Calcul des c_{ijk}
 - Assemblage des $K_{jk} = \sum_i^{OrdreKL} c_{ijk} K_i$
- Résolution :
 - Assemblage du terme source $\mathcal{F}$
 - Construction de la solution U
 - Résolution du système linéaire $\mathcal{K}U = \mathcal{F}$ en python à l'aide de PETSc.
 - Récupération et exportation des T_j
 - Code de réalisation d'une solution effective à partir des coefficients T_j

La programmation a donc été envisagée en Orientée Objet. Voici les différentes classes effectivement réalisées, et les objets principaux que contiennent ou calculent ces classes :

Classe KL :

- φ_i , paramètre de l'expansion KL. Calculés et mis à jour dans le .champs à chaque calcul et assemblage de K_i .
- λ_i , paramètre de l'expansion KL. Calculés et mis à jour dans le .champs à chaque calcul et assemblage de K_i .

Classe PC :

- α , liste d'entiers permettant de définir de manière unique chaque fonction de la base PC.
- c_{ijk} , coefficients calculés à partir de la méthode de minimisation d'erreur de Galerkin, $c_{ijk} = E[\xi_i \psi_j \psi_k]$

Dans le code SSFEM python :

- K_i , matrice de rigidité PETSc, calculée et assemblée dans MEFPP pour chaque terme de l'expansion KL, et récupérée dans python.

- F , vecteur de résidu PETSc, calculé et assemblé dans MEFPP pour le terme 0 de l'expansion KL, équivalent au vecteur de résidu du problème déterministe.
- K , matrice matnest PETSc, calculée et assemblée dans python à l'aide des K_i et des c_{ijk}
- $\mathcal{F}$, vecteur vecnest PETSc, assemblé dans python avant la résolution du système linéaire final.
- U , vecteur vecnest PETSc, construit dans python avant la résolution du système linéaire final.

Le développement du code s'est fait dans l'ordre du plan de développement.

0.3.5 Implémentation de la méthode

Une fois la méthode prise en main et le plan de développement mis sur papier, nous nous sommes ensuite lancés dans l'implémentation du code final.

A ce stade, nous possédons plusieurs fichiers python que nous importerons en tant que bibliothèques :

- `kl_expansion` bibliothèques implémentant l'expansion KL, permettant de calculer les λ_i
- `kl_varphi` permettant de lire les fonctions φ_i dans le fichier `.champs`
- `p_chaos` implémentant l'expansion PC, permettant notamment de calculer les c_{ijk}
- `ks_stat` implémentant les test de kolmogorov smirnov, qui nous serviront dans un second temps, pour la validation des résultats.

La méthode principale a donc été implémentée dans une classe nommée `ssfem`, qui possède plusieurs méthodes :

- `init` Cette méthode initialise le problème mefpp. Elle lit les $\varphi_{i=0}^{15}$ dans le fichier `.champs`, récupère les pré-post traitements définis dans le champ, récupère les matrices de rigidité et de résidu initialisées dans le `.champs`, et initialise une liste `l_matK` qui permettra de stocker les différentes matrices de rigidité calculées pour chaque terme de l'expansion KL.
- `assemblage_premier` Cette méthode permet de récupérer les matrices de rigidité du problème pour chaque terme de l'expansion. Ainsi, pour $i = 0 \dots \text{ordreKL}$, le fichier `.champs` résout le problème :

$$\mathbf{k}_i^e = \sqrt{\lambda_i} \int_{\Omega_e} \varphi_i(x) \mathbf{B}^T \cdot \mathbf{D}_0 \cdot \mathbf{B} d\Omega_e$$

puis assemble chaque $\mathbf{k}_i^e$ en K_i . Les K_i sont ainsi stockées dans `l_matK`

dans l'ordre.

- **assemblage_second** Cette méthode réalise le second assemblage des matrices de rigidité. Elle réalise l'opération :

$$\mathbf{K}_{jk} = \sum_{i=0}^M c_{ijk} \mathbf{K}_i$$

avec $j, k = 0, \dots, \text{ordrePC}$ et stocke les $\mathbf{K}_{jk}$ dans la matrice de matrice $\mathcal{K}$.

- **assemblage_F** Cette méthode construit le résidu F final, tel que :

$$\begin{bmatrix} F_0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

- **construction_T** Cette méthode construit la matrice de la solution finale T , pour qu'elle ait la bonne taille.
- **resolution_lineaire** Cette méthode réalise la résolution du système final $\mathcal{KT} = \mathcal{F}$. Elle initialise un solveur PETSc et réalise la résolution itérative. La solution est stockée dans la matrice T construite par **construction_T**. Il est à noter que les matrices de résidu et rigidité assemblées par MEFPP prennent déjà en compte les conditions aux bord du problème. Ainsi, il n'est pas nécessaire de réaliser une étape supplémentaire de prise en compte de ces critères, comme dans [10].
- **realiser** A partir de la solution calculée par **resolution_lineaire**, nous tirons un échantillon $\xi_i(\theta)_{i=0}^M$, avec $\xi_i \rightarrow N(0, \sigma^2)$ et réalisons l'étape d'assemblage :

$$\mathbf{T}(\theta) = \sum_{j=0}^P \mathbf{T}_j \psi_j(\theta)$$

que nous stockons dans un attribut **self.realisation** de la classe, qui est un vecteur PETSc.

- **exportation** Cette méthode exporte l'attribut **self.realisation** en le copiant dans un champ MEFPP et en l'exportant à partir d'un prépost traitement défini également dans le champ.
- **mc** La méthode Monte Carlo exécute n_mc fois la méthode **realiser** et stocke les valeurs de la solution T en un noeud fixé, pour chaque réalisation, dans une liste.

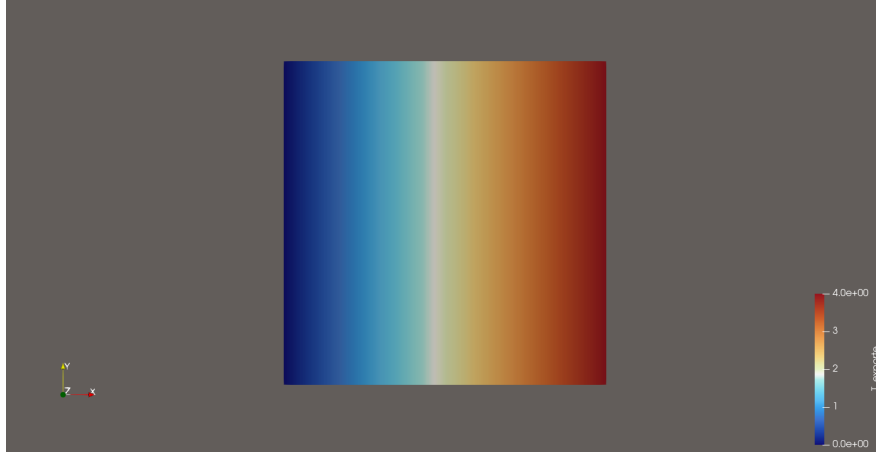


FIGURE 9 – Solution déterministe cas $D(x, \theta) = 1$

- **test_ks** Cette méthode vérifie le fit des tirages Monte Carlo avec une loi normale $N(\mu, \sigma^2)$ avec μ la valeur de la solution déterministe au noeud considéré, et σ la variance effective des réalisations en ce noeud

0.3.6 Validation de la méthode

Une fois le code implémenté, il est nécessaire de valider les résultats obtenus. Tout d'abord, nous allons considérer un nouveau problème dont la solution est simple. Considérons un problème de conduction thermique en 1 dimension.

$$\begin{cases} \nabla (D \nabla T) &= 0 \\ T_{\text{bordgauche}} &= 0 \\ D \nabla T_{\text{bordgauche}} &= 1 \\ D \nabla T_{\text{borddroit}} &= 1 \end{cases}$$

Pour des problèmes d'implémentation en MEFPP, la géométrie de l'espace du problème est un carré en 2 dimension, dans Oxy , mais les variables et paramètres seront indépendants sur Oy . Ainsi, nous nous ramenons à la solution en 1D.

Nous prendrons ici $D(x, \theta)$ le champ aléatoire, avec $D_0 = 1$ la partie déterministe de ce champ. Dans ce cas, la solution exacte de ce problème est $T =$.

Partie déterministe

La première étape est la validation du calcul de la partie déterministe de la solution. Puisque $U(\theta) = \sum_{j=0}^P \psi_j(\theta) U_j$ et puisque les ψ_j vérifient $E[\psi_j(\theta)] = 0$,

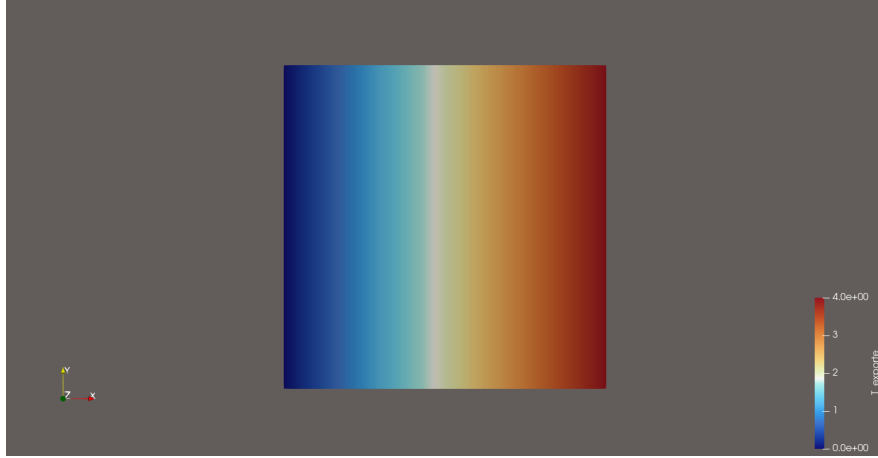


FIGURE 10 – Solution SSFEM ordreKL = 0, ordrePC = 0

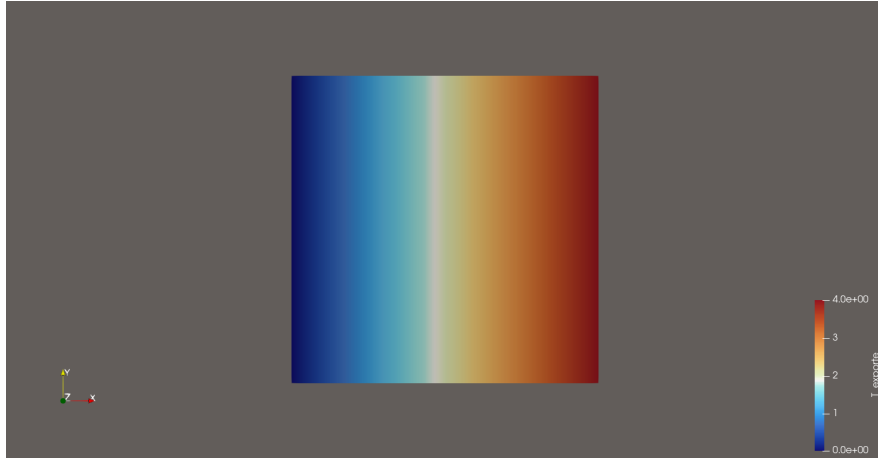


FIGURE 11 – Solution SSFEM ordreKL = 5, ordrePC = 1

alors le vecteur moyen de température, qui est aussi la solution dans le cas déterministe, par linéarité du problème, est T_0 . Ainsi, $\forall \text{ordreKL} \& \text{ordrePC} \quad T_0 = T_{\text{deterministe}}$.

- Pour ordreKL = 0 ordrePC = 0, la solution est numériquement exacte.
- Pour ordreKL = 5 ordrePC = 1, la solution présente une erreur maximum $e = 0.004$.
- Pour ordreKL = 5 ordrePC = 10, la solution dévie, et présente une erreur allant jusqu'à $e = 3.5$
- Pour ordreKL = 14 ordrePC = 1, la solution présente une erreur maximum $e = 0.0003$
- Pour ordreKL = 14 ordrePC = 10 la solution présente une erreur maxi-



FIGURE 12 – Solution SSFEM ordreKL = 14, ordrePC = 10

mum $e = 0.007$.

Nous avons choisi un problème simple à visualiser, mais il est difficile d’observer l’impact des variations du champs D sur la solution. Il est à noter qu’il serait plus judicieux de visualiser l’écart entre la réalisation aléatoire et la solution déterministe.

Nous observons donc que la partie déterministe de la solution SSFEM s’approche de la solution exacte lorsqu’on augmente ordreKL et ordrePC. Néanmoins, lorsque $\text{ordrePC} \geq \text{ordreKL}$, la solution déterministe dévie de la solution exacte, et la tendance n’est même plus linéaire.

Nous pouvons conclure que la partie déterministe SSFEM converge vers la solution exacte.

Dans la suite, nous considérerons donc le plus grand ordreKL = 14 implémenté, et ordrePC = 10.

Test Kolmogorov Smirnov

Désormais, puisque la partie déterministe SSFEM a été validée, nous devons désormais nous intéresser à la partie aléatoire.

Nous allons donc réaliser un post traitement statistique sur la solution. Choisissons un noeud arbitraire, numéroté 64. Nous noterons T_{64} la valeur de la solution PC sur ce noeud. Nous allons tout d’abord vérifier le test de Kolmogorov Smirnov sur ce noeud, en comparant la loi suivie par les valeurs de T en ce noeud, avec la loi normale centrée sur la valeur déterministe SSFEM en ce noeud.

La première chose que nous observons est que désormais, l'espérance et la variance Monte Carlo dépendent du paramètre r de l'expansion KL.

- Pour $r = 0.26$, le test de Kolmogorov Smirnov fit la loi suivie par T_{64} avec $\mathcal{N}(-0.08, 0.2)$ environ.
- En ajustant r , nous trouvons que pour $r = 0.12$, le test de Kolmogorov Smirnov fit la loi suivie par T_{64} avec $\mathcal{N}(2.5, 0.045)$.

Néanmoins, la représentation PC de $T_{64} = \sum_{j=0}^P T_{j64} \psi_j(\theta)$ avec les $\psi_j \rightarrow \mathcal{N}(0, 1)$. Donc $E[T_{64}] = T_{\text{exacte}} = 2.5$. Ainsi, contrairement à ce qui est stipulé dans [7], le paramètre r n'est pas libre, et a un impact sur la méthode. Il est à noter que nous ne pouvons pas modifier la loi suivie par les θ car la base PC ne serait alors plus orthonormale.

Comparaison SSFEM et champ convolué

Nous souhaiterions désormais comparer la loi de T_{64} SSFEM et dans le cas où E est convolué spatialement. Néanmoins, la géométrie du problème ayant été réalisée en 2 dimensions, le champ D convolué est donc convolué en 2 dimensions également. Ainsi, il faut adapter soit la convolution, soit le problème SSFEM en 1 dimension.

Nous souhaiterions donc passer en 2 dimensions. Cela signifie qu'il faut considérer l'expansion KL pour une fonction à valeurs dans $\Omega \times \Omega$. D'après [11], le cas bidimensionnel pour l'expansion KL peut être obtenu à partir du cas unidimensionnel, en prenant $\lambda_i \rightarrow \lambda_i^2$ et $\varphi_i(x) \rightarrow \varphi_i(x)\varphi_i(y)$. Néanmoins, il est nécessaire d'avoir un matériau anisotrope. En prenant $x = (x_1, x_2) \in \Omega \times \Omega$, il faudrait donc $\text{Cov}(x, x') = \text{Cov}(x_1, x_2)\text{Cov}(x'_1, x'_2)$, et malheureusement non pas $\text{Cov}(x, x') = \exp\left(\frac{-(\text{norm}(x-x'))^2}{2w^2}\right)$. Dans ce cas d'anisotropie, il faudrait donc effectuer une convolution également anisotrope, ce qui reviendrait en quelque sorte à devoir réaliser une convolution unidimensionnelle, sur l'axe Ox et Oy .

C'est à cette étape de la démarche que le stage s'est clôturé

0.4 Pistes à explorer et travaux futurs

Ce stage de recherche au sein du laboratoire du GIREF a duré 12 semaines. Malheureusement, toutes les problématiques soulevées n'ont pu complètement aboutir. La partie aléatoire du code SSFEM, notamment, n'a pas pu être vérifiée. Certains résultats, tels que l'écart théorique et expérimental pour la fonction de covariance post convolution, ou encore le facteur 50 dans le fit de la variance pour le test de Kolmogorov Smirnov, n'ont pas eu le temps d'être expliqués. A la lumière de cette réflexion, voici quelques pistes envisageables pour approfondir ces travaux :

champ aléatoire lognormal

Dans un critère de réalisme de la modélisation du module de Young, il est impossible que ce module soit négatif en un point. Cependant dans ces travaux, il a toujours été fait mention de la loi normale $\mathcal{N}$.

Ainsi, il est plus réaliste pour modéliser le module de Young de considérer une loi lognormal, par souci de positivité de E . [10] Le cas d'un champ lognormal est décrit dans la littérature. Il faut dans ce cas remplacer l'expansion KL par l'expansion PC.

p-value test Kolmogorov Smirnov

Dans un souci de validation des hypothèses vérifiées par le test de Kolmogorov Smirnov, il est utile voire nécessaire de calculer la p-value associée. Cela n'a pas été abordé mais il existe des packages prenant cela en compte. Il existe également une formule fermée pour la p-value à n la taille de l'échantillon grand.

Méthode des perturbation

Également, il peut être intéressant d'étudier la méthode des perturbations.

0.5 Problèmes rencontrés

Plusieurs problèmes ont été rencontrés lors de la réalisation de ce stage. Tout d'abord, bien que des efforts soient mis en place pour la prise en main de MEFPP, certaines particularités ou fonctionnalités sont peu ou pas documentées. Ainsi, il est alors nécessaire de se référer au code source, en trouvant les bons fichiers et en comprenant l'implémentation des fonctions.

Autrement, certaines fonctionnalités n'étant pas implémentées, nous devons trouver des astuces pour que le code réalise ce que l'on souhaite. C'est le cas par exemple lors de l'exportation en mefpp de champ, obtenus ou modifiés en python. Il est nécessaire de passer par des problèmes factices pour obtenir un champ dont les coefficients sont numérotés "conformément" à la géométrie du problème.

Un autre problème majeur a été rencontré lors de l'implémentation KL. En effet, les variables décrivant la résolution du problème de valeurs propres pour la fonction $\exp(-x^2)$ sont différentes en fonction des articles [3] [7]. Voici la description dans le premier article : Pour la fonction de covariance :

$$e^{-\varepsilon^2(x-z)^2} = \sum_{n=0}^{\infty} \lambda_n \varphi_n(x) \varphi_n(z),$$

Nous avons :

$$\lambda_n = \frac{\alpha \varepsilon^{2n}}{\left(\frac{\alpha^2}{2} \left(1 + \sqrt{1 + \left(\frac{2\varepsilon}{\alpha} \right)^2} \right) + \varepsilon^2 \right)^{n + \frac{1}{2}}},$$

$$\varphi_n(x) = \frac{\sqrt{1 + \left(\frac{2\varepsilon}{\alpha} \right)^2}}{2^n n!} e^{-(\sqrt{1 + \left(\frac{2\varepsilon}{\alpha} \right)^2} - 1) \frac{\alpha^2 x^2}{2}} H_n \left(\sqrt{1 + \left(\frac{2\varepsilon}{\alpha} \right)^2} \alpha x \right)$$

Ainsi nous avons trouvé la relation entre les 2 articles, avec $\alpha = f(\sigma)$. Les autres relations entre les paramètres s'en déduisent. Néanmoins, cela signifie qu'il n'y a pas de paramètre libre dans cette écriture de l'expansion. Qui plus est, lorsque la description de [3] a été implémentée en Python, nous observions que la variance du champ aléatoire $\{\text{Var}(x_i, x_i)\}_{i \leq N}$ divergeait lorsqu'on augmentait l'indice de troncature de l'expansion, à part pour une valeur $\alpha = \sqrt{2}$.

Il aurait donc fallu se pencher sur la démonstration entière, à base de la formule de Mehler, mais par faute de temps, cela n'a pas été entamé.

0.6 Conclusion

Le logiciel MEFPP présente beaucoup d'intérêts pour la résolution de problèmes physiques par méthode des éléments finis. La première est sa facilité d'utilisation. Bien que la prise en main soit fastidieuse, beaucoup de documentation est mise en place pour faciliter cette prise en main. Avec la bibliothèque `mefpp4py` nouvellement développée, il est très facile de réaliser des analyses statistiques post-résolution.

Ainsi, le code de simulation Monte Carlo pour le problème de cisaillement est facile d'utilisation, est permet de réaliser plusieurs centaines de compilations en moins de quelques minutes (sur une géométrie et un problème ne possédant qu'environ mille degrés de liberté). Le problème dépend linéairement du champ E lorsque celui-ci est constant. Néanmoins, en prenant l'exemple unidimensionnel, le problème $\nabla(E(x)\nabla u(x)) = f(x)$ est plus compliqué dès lors que E dépend de l'espace.

D'autre part, la méthode SSFEM présente des intérêts computationnels. En effet, elle nécessite la résolution d'un système de matrices de matrices, d'autant plus grand que les ordres d'expansion augmentent. Néanmoins, une fois la solution de ce système calculée, les simulations et analyses sont directes et, contrairement à la méthode force brute de Monte Carlo, ne nécessitent pas de résolution de système. Néanmoins, certaines limitations sur la méthode SSFEM existent [10], notamment :

- Méthode limitée aux problèmes linéaires
- Aucune méthode de calcul de l'erreur due à la troncature n'a été explicitée dans la littérature

Un code fonctionnel de la méthode SSFEM a donc été implémenté en python et MEFPP. Ce code a été validé pour la partie déterministe et semble cohérent pour la partie aléatoire, mais une validation reste nécessaire pour confirmer la validité du code et de la méthode

Bibliographie

- [1] Chatwin ANDERSON Barnett. *Quantitative methods*.
- [2] Björn ENGQUIST. *Encyclopedia of Applied and Computational Mathematics*. Sous la dir. de Björn ENGQUIST. SpringerReference, 2015.
- [3] Gregory E. FASSHAUER. « Positive Definite Kernels : Past, Present and Future ». In : *Illinois Institute of Technology* 1.1 (2000).
- [4] André Fortin André GARON. *Les éléments finis en mécanique des milieux continus*. Université Laval, Québec, 2025.
- [5] Barry Ver HOEF. « Blackbox Kriging : Spatial Prediction Without Specifying Variogram Models ». In : *Journal of Agricultural Biological and Environmental Statistics* 1.3 (1996).
- [6] KH Lee Higdon Calder HOLLOMAN. « Efficient Models for Correlated Data via Convolutions of Intrinsic Processes ». In : *School of Engineering, UCSC 1156 High Street Santa Cruz* (2004).
- [7] JESAISPAS. « Gaussian Kernel on 1D Gaussian data eigendecomposition ». In : *Jesaispas* 0.0 (0). <https://james-simon.github.io/blog/gaussian-kernel-1d-eigendecomposition/>
- [8] Sudret KIUREGHIAN. « Comparison of finite element reliability methods ». In : *Probabilistic Engineering Mechanics* 0.17 (2002).
- [9] Ghanem & SPANOS. *Stochastic Finite Elements : A Spectral Approach*. 1991.
- [10] SUDRET. « Stochastic Finite Element Methods and Reliability ». In : *University of California* 1.1 (2000), p. 189.
- [11] Rasmussen WILLIAMS. *Gaussian Processes for Machine Learning*. Sous la dir. de MIT PRESS. 2006.

Glossaire

Finite Element Method Méthode de résolution des équations différentielles par maillage de l'espace, puis discrétisation des paramètres sur cet espace, et projection de la solution théorique sur un espace de dimension finie.. 39

Groupe Interdisciplinaire de Recherche en Éléments Finis Laboratoire spécialisé en Éléments finis, dépendant du département de Mathématiques et Statistiques de l'Université Laval. ii, iii, 2

MEFPP Logiciel de simulation numérique utilisant la méthode des éléments finis pour résoudre des problèmes variés. ii, iii

mefpp4py Bibliothèque développée en C++, permettant d'interfacer MEFPP et Python. mefpp4py permet d'interagir entre le langage python et les fichiers .champs de configuration MEFPP.. 5

Polynomial Chaos Base de l'espace des variables aléatoires mesurables $\mathcal{L}^2(\Theta, \mathcal{F}, P)$. C'est sur cette base que l'on discrétise la solution dans la méthode SSFEM. 40

pp Pré-post traitements. Fonctions précodées disponibles sur MEFPP, généralement exécutées avant, pendant, ou après la résolution du problème d'éléments finis. Elles permettent : d'exporter les résultats, de modifier les valeurs des champs, d'interpoler des champs 6

Spectral Stochastic Finite Element Method Méthode numérique de résolution de systèmes d'équations différentielles stochastique. Il s'agit d'une extension de la FEM. ii

Uncertainty quantification Évaluation des incertitudes sur les paramètres d'intérêt d'un problème, dont les caractéristiques d'entrée présentent une incertitude.. ii, iii

0.7 Annexe

0.7.1 Visualisation des matrices c_{ijk} pour différents ordres, à troncatures KL et PC fixées.

```
>>> Matrice_c0_3_10
array([[1., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
       [0., 1., 0., 0., 0., 0., 0., 0., 0., 0.],
       [0., 0., 1., 0., 0., 0., 0., 0., 0., 0.],
       [0., 0., 0., 1., 0., 0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 2., 0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0., 1., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0., 0., 2., 0., 0., 0.],
       [0., 0., 0., 0., 0., 0., 0., 1., 0., 0.],
       [0., 0., 0., 0., 0., 0., 0., 0., 1., 0.],
       [0., 0., 0., 0., 0., 0., 0., 0., 0., 2.],
       [0., 0., 0., 0., 0., 0., 0., 0., 0., 0.]])
```

FIGURE 13 – Matrice des c_{ijk} pour $M = 3, P = 10, i = 0$

```
>>> Matrice_c1_3_10
array([[0., 0., 1., 0., 0., 0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0., 1., 0., 0., 0., 0.],
       [1., 0., 0., 0., 0., 0., 2., 0., 0., 0.],
       [0., 0., 0., 0., 0., 0., 0., 1., 0., 0.],
       [0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
       [0., 1., 0., 0., 0., 0., 0., 0., 0., 0.],
       [0., 0., 2., 0., 0., 0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
       [0., 0., 0., 1., 0., 0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0., 0., 0., 0., 0., 0.]])
```

FIGURE 14 – Matrice des c_{ijk} pour $M = 3, P = 10, i = 1$

```
>>> Matrice_c2_3_10
array([[0., 1., 0., 0., 0., 0., 0., 0., 0., 0.],
       [1., 0., 0., 0., 2., 0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0., 1., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0., 0., 1., 0., 0., 0.],
       [0., 2., 0., 0., 0., 0., 0., 0., 0., 3.],
       [0., 0., 1., 0., 0., 0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
       [0., 0., 0., 1., 0., 0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 3., 0., 0., 0., 0., 0.]])
```

FIGURE 15 – Matrice des c_{ijk} pour $M = 3, P = 10, i = 2$

0.7.2 Aberration de résolution lorsque $\text{ordrePC} \geq \text{ordreKL}$

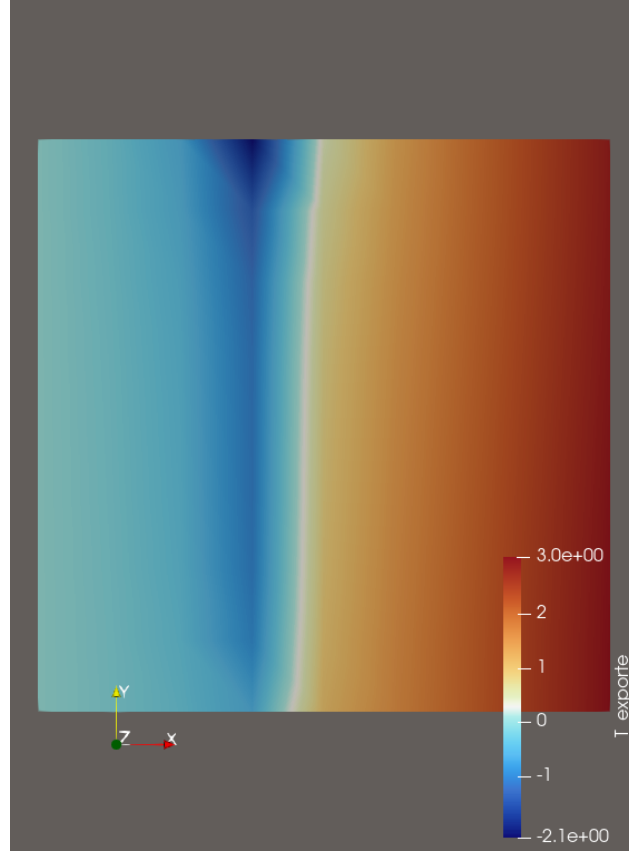


FIGURE 16 – Solution SSFEM $\text{ordreKL} = 5, \text{ordrePC} = 10$

Nous observons que la solution semble dépendre de y . Cette aberration peut être due au fait que le problème est unidimensionnel de manière factice.

0.7.3 Détail SSFEM

Nous allons ici détailler la méthode SSFEM [10], en considérant la méthode Finite Element Method comme acquise et connue. La méthode des éléments finis, qui est la base théorique de MEFPP, est détaillée dans [4].

Considérons un problème d'élasticité linéaire en 2 dimensions. Le problème Éléments Finis assemblé, qui est de taille $N \times N$ avec N le nombre de degré de liberté (nombre de sommets du maillage $\times$ nombre de grandeurs d'intérêts), s'écrit :

$$\mathbf{K} \cdot \mathbf{U} = \mathbf{F}$$

avec $\mathbf{K}$ la matrice de rigidité du système, assemblée à partir de la matrice élémentaire

$$\mathbf{k}^e = \int_{\Omega_e} \mathbf{B}^T \cdot \mathbf{D} \cdot \mathbf{B} d\Omega_e$$

sur un sous-espace élémentaire Ω_e de référence, de l'espace Ω . $\mathbf{D}$ est la matrice d'élasticité du matériau et $\mathbf{B}$ la matrice reliant la contrainte aux déplacements (tenseur de cauchy).

Dans le cas stochastique, nous considérons $\mathbf{D}$ comme une matrice de champ aléatoire, $\mathbf{D}(x, \theta) = H(x, \theta)\mathbf{D}_0$, H étant un champ aléatoire, D_0 la matrice d'élasticité moyenne, dans le cas déterministe. Ainsi, en utilisant l'expansion KL sur H :

$$H(x, \theta) = \mu(x) + \sum_{i=1}^{\infty} \sqrt{\lambda_i} \varphi_i(x) \xi_i(\theta)$$

avec φ_i des fonctions $\mathcal{L}^2$ mesurables, et ξ_i des variables aléatoires $\sim \mathcal{N}(0, 1)$.

On peut ainsi décomposer à l'aide de l'expansion kl : $\mathbf{k}^e(\theta) = \mathbf{k}^e + \sum_{i=1}^{\infty} \mathbf{k}_i^e \xi_i(\theta)$ avec

$$\mathbf{k}_i^e = \sqrt{\lambda_i} \int_{\Omega_e} \varphi_i(x) \mathbf{B}^T \cdot \mathbf{D}_0 \cdot \mathbf{B} d\Omega_e$$

En assemblant chaque $\mathbf{k}_i^e$ en $\mathbf{K}_i$, comme dans le cas déterministe, nous obtenons le système :

$$\left[\mathbf{K}_0 + \sum_{i=1}^{\infty} \mathbf{K}_i \xi_i(\theta) \right] \mathbf{U}(\theta) = \mathbf{F}$$

.

De la même manière, puisque la solution $\mathbf{U} = \mathbf{U}(x, \theta)$ est aléatoire, nous la décomposons dans une base de $\mathcal{L}^2(\Theta, \mathcal{F}, P)$, la Polynomial Chaos polynomial chaos. En supposant donc que nous pouvons l'étendre dans PC :

$$\mathbf{U}(\theta) = \sum_{j=0}^{\infty} \mathbf{U}_j \psi_j(\theta)$$

avec ψ_j les fonctions PC centrées et ordonnées par ordre croissant de degré, $\{\psi_j(\theta)\}_j$ une base orthogonal de $\mathcal{L}^2(\theta, \mathcal{F}, P)$. Les $\mathbf{U}_j$ sont des vecteurs de taille N , qui représentent des coordonnées déterministes de $\mathbf{U}$ dans la base $\mathcal{L}^2(\theta, \mathcal{F}, P)$.

A la suite de ces deux expansions, nous nous retrouvons donc avec le système :

$$\left(\sum_{i=0}^{\infty} \mathbf{K}_i \xi_i(\theta) \right) \cdot \left(\sum_{j=0}^{\infty} \mathbf{U}_j \Psi_j(\theta) \right) - \mathbf{F} = 0$$

Pour des raisons numériques, il est nécessaire de tronquer ces séries. Nous tronquons donc l'expansion KL à l'ordre M ($M + 1$ termes dans la série) et l'expansion PC à l'ordre P ($P + 1$ termes dans la série). Nous obtenons un résidu dû à la troncature :

$$\epsilon_{M,P} = \sum_{i=0}^M \sum_{j=0}^{P-1} \mathbf{K}_i \cdot \mathbf{U}_j \xi_i(\theta) \Psi_j(\theta) - \mathbf{F}$$

La solution $\mathbf{U}$ de ce système est donc obtenue en minimisant le résidu. Il s'agit désormais, pour résoudre le problème, d'une méthode de minimisation de Galerkin de ce résidu, dans l'espace engendré par les $\{\psi_j\}_{j=0}^P = \mathcal{H}_P$. Cela revient donc à avoir un résidu orthogonal à $\mathcal{H}_P$. En projetant l'équation sur chaque ψ_j , $\forall j = 0 \dots P$, nous obtenons le système :

$$\sum_{i=0}^M \sum_{j=0}^{P-1} c_{ijk} \mathbf{K}_i \cdot \mathbf{U}_j = \mathbf{F}_k, \quad k = 0, \dots, P-1$$

avec

$$c_{ijk} = \mathbb{E}[\xi_i \Psi_j \Psi_k]$$

$$\mathbf{F}_k = \mathbb{E}[\Psi_k \mathbf{F}]$$

Il existe une formule fermée permettant de calculer les c_{ijk} directement à partir des coefficients i, j, k , une fois que les polynômes ψ_j ont été implémentées. Le calcul est détaillé dans [10]. Finalement, en définissant $\mathbf{K}_{jk} = \sum_{i=0}^M c_{ijk} \mathbf{K}_i$, nous obtenons un système affine :

$$\begin{bmatrix} K_{00} & \cdots & K_{0,P-1} \\ K_{10} & \cdots & K_{1,P-1} \\ \vdots & \ddots & \vdots \\ K_{P-1,0} & \cdots & K_{P-1,P-1} \end{bmatrix} \begin{bmatrix} U_0 \\ U_1 \\ \vdots \\ U_{P-1} \end{bmatrix} = \begin{bmatrix} F_0 \\ F_1 \\ \vdots \\ F_{P-1} \end{bmatrix}$$

Que l'on réécrit :

$$\mathcal{K}\mathcal{U} = \mathcal{F}$$

Ainsi la solution finale est $\mathbf{U}(\theta) = \sum_{j=0}^P \mathbf{U}_j \psi_j(\theta)$,

La solution du système est donc $(\mathbf{U}_j)_j$ dont chaque élément est lui-même un vecteur de taille N . A partir de ces vecteurs, il est donc possible de mener une analyse post traitement statistique.